

# Resend Email Setup — Cinder + BYB

## Resend email setup (Cinder + BYB)

---

**What is Resend?** Resend is the *email-sending service* the apps plug into to deliver the email they generate themselves — login codes, invites, password resets. It's developer infrastructure (an API + SMTP), in the same family as SendGrid / Postmark / Amazon SES. It is **not** a marketing tool like Mailchimp (campaigns, newsletters, drag-and-drop builders) — it's the behind-the-scenes plumbing that makes "transactional" email actually arrive in the inbox.

**Goal:** make the apps able to send real emails (login OTP codes, workspace invites, email confirmations) to *any* user — not just members of the Supabase org.

This guide covers **two separate projects** that each need their own email setup:

- **Cinder** — the network-marketer pipeline CRM (Supabase project `swrxrtifmygripfxdemh`).
- **BYB** ("Build Your Guild" — the AU/NZ business-OS, repo `byb-platform`, Supabase project `zoqhmsscpfsgngatykuro`). BYB handles money + bank data and is held to a **bank-grade security standard**, so its email setup has extra hardening — see Part 5.

⚠ **Keep the two projects' email fully separate.** One Resend account is fine, but **each project gets its own verified domain (or subdomain) and its own API key**. Never reuse Cinder's domain or key for BYB (or vice-versa): separate sending reputation, independent rotation/revocation, and — for BYB — its bank-grade standard *requires* secret isolation (no shared keys across systems).

**Why this is needed:** both hosted Supabase projects are currently on Supabase's **default built-in mailer**, a shared, heavily rate-limited service that **only delivers to addresses belonging to the Supabase org**. So today, if a real prospect/recruit/client tries to log in with email OTP, or you send them an invite, the email silently never arrives. Wiring a real SMTP provider (Resend) fixes it.

# Per-project reference

|                          | Cinder                                  | BYB                                                      |
|--------------------------|-----------------------------------------|----------------------------------------------------------|
| Supabase project ref     | swrxrtifmygripfxdemh                    | zoqhmsscpcfsgngatykuro                                   |
| Org                      | Delilah hfmgyospmdymoiwjewu             | Delilah hfmgyospmdymoiwjewu (same)                       |
| Suggested sending domain | mail.cinder.app (subdomain)             | mail.<byb-domain> (its own domain — <b>not</b> cinder's) |
| Suggested from-address   | login@mail.cinder.app                   | noreply@mail.<byb-domain>                                |
| Resend API key name      | cinder-supabase-auth                    | byb-supabase-auth + byb-app-invites (two — see Part 5)   |
| Email paths              | Supabase Auth only                      | Supabase Auth + Express server app-invites (Part 5)      |
| Railway                  | valiant-rejoicing / content-flexibility | diligent-enthusiasm / server + byb-web                   |

**Heads-up (org-role ceiling — applies to BOTH projects):** the Supabase-side steps (Part 3) change project Auth settings. Belinda is a **non-owner member** of Delilah's org, so the CLI `supabase config push` returns 403 and the dashboard SMTP settings may be read-only for her. **Delilah (org owner)** likely has to do Part 3 for each project — but Belinda can do *all* of Part 1 + Part 2 herself (create the Resend account, verify each domain, generate the keys) and just hand the keys + sender addresses to Delilah.

# TL;DR — the domain question, answered first

"Does the from-address need to be a domain, or can it be one specific email for the business?"

**You need a domain. You cannot authorize a single arbitrary inbox (a Gmail/iCloud/Outlook address) as a sender.** That's not a Resend quirk — it's how email authentication (SPF/DKIM) works everywhere: trust is proven at the **domain** level by adding DNS records to a domain *you control*. Once a domain is verified, you can then send from **any** mailbox at that domain.

Three concrete choices:

| Option                                                         | Sender example                         | Use when                                                                                                |
|----------------------------------------------------------------|----------------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>A. Verify a domain you own</b> ( <i>production</i> )        | hello@cinder.app ,<br>login@cinder.app | Real launch. Best deliverability + branding.                                                            |
| <b>B. Dedicated sending subdomain</b> ( <i>best practice</i> ) | login@mail.cinder.app                  | Same as A, but isolates email reputation from your root domain.<br><b>Recommended.</b>                  |
| <b>C. Resend's test sender</b> ( <i>throwaway</i> )            | onboarding@resend.dev                  | Quick smoke test only — <b>delivers ONLY to your own Resend-account email</b> , useless for real users. |

So the practical answer: **buy/own a domain for Cinder** (e.g. `cinder.app`, `trycinder.com`, `getcinder.com` — Cinder has no custom domain yet; the web app currently runs on a `*.railway.app` URL), then pick **one specific from-address at that domain** (e.g. `login@cinder.app`). You're not verifying the single address — you're verifying the domain, then choosing an address on it.

If you don't yet own a Cinder domain, you can do Part 1–2 with the test sender (Option C) to confirm the plumbing works, then swap in the real domain before launch.

## Part 1 — Create the Resend account & API key

- Go to <https://resend.com> → sign up (free tier = 3,000 emails/month, 100/day — plenty for OTP/invite volume early on).
- In the dashboard, go to **API Keys** → **Create API Key**.
  - Name: `cinder-supabase-auth`
  - Permission: **Sending access** (it doesn't need full access).
  - Copy the key ( `re_...` ) **now** — it's shown once. This is the SMTP password later.

## Part 2 — Verify your sending domain

Skip to Part 3 with `onboarding@resend.dev` if you just want to test plumbing first.

- Resend dashboard → **Domains** → **Add Domain**. Enter your domain. **Recommended:** enter a subdomain like `mail.cinder.app` (not the bare `cinder.app`) so auth email reputation stays separate from anything you send from the root domain.
- Resend shows a set of **DNS records** to add at your domain registrar / DNS host (e.g. Porkbun, Cloudflare, Namecheap):
  - MX + TXT (SPF)** — for the sending subdomain.
  - 3× CNAME (DKIM)** — `resend._domainkey...` records that sign your mail.

- (Optional but recommended) a **DMARC** TXT record at `_dmarc.<domain>: v=DMARC1; p=none; rua=mailto:you@yourdomain`
3. Add each record exactly as shown in your DNS host, then click **Verify** in Resend. DNS can take minutes–hours to propagate; the domain flips to **Verified** when ready.
  4. Once verified, you can send from **any** address at that domain. Pick a clear one for Cinder, e.g.:
    - `login@mail.cinder.app` (OTP/auth), or
    - `hello@cinder.app` (friendlier, single address for everything).

## Part 3 — Point Supabase Auth at Resend (SMTP) (owner / Delilah)

---

This is the step that needs org-owner access (Belinda's `config push` 403s). Do it in the **Supabase Dashboard** for project **Cinder** (`swrxrtifmygripfxdemh`):

1. **Authentication** → **Emails** → **SMTP Settings** → toggle **Enable Custom SMTP** on. Enter:

| Field        | Value                                                                     |
|--------------|---------------------------------------------------------------------------|
| Host         | <code>smtp.resend.com</code>                                              |
| Port         | <code>465</code> (SSL) — or <code>587</code> (STARTTLS) if 465 is blocked |
| Username     | <code>resend</code>                                                       |
| Password     | the Resend API key from Part 1 ( <code>re_...</code> )                    |
| Sender email | your verified-domain address, e.g. <code>login@mail.cinder.app</code>     |
| Sender name  | <code>Cinder</code>                                                       |

2. **While you're in here, fix the three known config drifts** (the hosted project is on defaults that diverge from the repo's `supabase/config.toml`):

- **OTP length:** set to **6** (hosted is currently **8**, but the mobile Verify screen renders **6** code boxes → 8-digit codes break email-OTP login). → *Authentication* → *Providers* → *Email* → *OTP length*.
- **Site URL:** set to the production web URL (currently `https://content-flexibility-production-55ea.up.railway.app`, or your custom domain once you have one) instead of `localhost`, so invite/confirmation links resolve. → *Authentication* → *URL Configuration*.
- **Email rate limit:** the default with custom SMTP is **2 emails/hour** — far too low. Raise it (e.g. 30–100/hour) → *Authentication* → *Rate Limits*.
- (Decision) **Confirm email:** repo `config.toml` has `enable_confirmations = false` (users sign in via OTP without a separate confirm step). Leave it off unless you want a confirm gate.

3. (Optional) Customize the OTP / invite **email templates** under **Authentication** → **Emails** → **Templates** so they're Cinder-branded (burgundy #6B1F2E , flame mark, Instrument Serif headline).

## Part 4 — Test it

---

1. **Quick path (no domain):** with Option C sender set, trigger an OTP to *your own* Resend-account email and confirm it arrives.
2. **Real path:** from the Cinder mobile app, do email login with a normal external address (a personal Gmail, a friend's email). The 6-digit code should land in that inbox within seconds.
3. If nothing arrives: check **Resend** → **Logs** (shows accepted/bounced/delivered per message) and **Supabase** → **Logs** → **Auth** (shows send attempts + SMTP errors).

## Part 5 — BYB ("Build Your Guild") — a separate setup

---

BYB is a **different project** (repo `byb-platform` , Supabase `zoqhmsscpfsngatykuro` , Railway `diligent-enthusiasm`). It is actively wiring email on branch `feat/pr1-resend-email` . Do **not** share Cinder's domain or keys — set BYB up independently. BYB also has **two** outbound-email paths, not one:

1. **Supabase Auth** (login OTP / confirmations) — same mechanism as Cinder.
2. **App-level invite emails sent by the Express server** — BYB's onboarding/invites currently use a **console transport** (logs the email instead of sending it). Real invites need a real transport, which is what the `feat/pr1-resend-email` branch is for.

### 5a. Resend — reuse the account, new domain + new keys

In the **same** Resend account (Part 1), do the following **separately for BYB**:

1. **Add BYB's own sending domain** (Part 2) — e.g. a subdomain `mail.<byb-domain>` of whatever domain BYB ships under. **Do not use a `cinder.app` subdomain for BYB** — different product, different reputation, and bank-grade isolation. Add the SPF/DKIM/DMARC DNS records and verify.
2. **Generate TWO API keys** (so the two paths rotate/revoke independently — bank-grade: minimize blast radius):
  - `byb-supabase-auth` — used as the SMTP password for BYB's Supabase Auth (5b).
  - `byb-app-invites` — used by the Express server's Resend transport (5c). Both with **Sending access** only.

### 5b. BYB Supabase Auth SMTP (owner / Delilah)

Identical to Part 3, but in the dashboard for **project BYB** ( `zoqhmsscpfsngatykuro` ):

| Field        | Value                                |
|--------------|--------------------------------------|
| Host         | smtp.resend.com                      |
| Port         | 465 (or 587)                         |
| Username     | resend                               |
| Password     | the byb-supabase-auth key ( re_... ) |
| Sender email | noreply@mail.<byb-domain>            |
| Sender name  | BYB (or the client-facing brand)     |

Apply the same three config fixes here too (BYB hits the **same non-owner org ceiling** as Cinder — hosted Auth is on defaults until Delilah sets them): **OTP length** (match the app's OTP UI), **Site URL** → BYB's web URL ( byb-web-production.up.railway.app or its custom domain, not localhost ), and **email rate limit** (raise off the 2/hour custom-SMTP default).

### 5c. BYB Express server invite transport ( feat/pr1-resend-email )

BYB's invites are sent by the **Node/Express server**, not a Supabase edge function — so this path calls the **Resend HTTP API** (or the resend npm SDK) directly from the server, using the byb-app-invites key. Set it as a server env var on Railway (service server):

```
RESEND_API_KEY=re_...          # the byb-app-invites key (NOT the Supabase-Auth
one)
RESEND_FROM="BYB <noreply@mail.byb-domain>"
```

Then replace the console transport with a Resend send (SDK shape):

```
import { Resend } from "resend";
const resend = new Resend(process.env.RESEND_API_KEY);

await resend.emails.send({
  from: process.env.RESEND_FROM!,           // must be on the verified BYB domain
  to: [inviteeEmail],
  subject: "You've been invited to BYB",
  html: renderInviteEmail(invite),
});
```

Keep the console transport as a **dev/test fallback** (when RESEND\_API\_KEY is unset) so local runs don't need a live key.

## 5d. Bank-grade hardening (BYB only)

BYB handles financial + bank data, so its email goes beyond Cinder's baseline:

- **Dedicated sending subdomain** ( `mail.<byb-domain>` ) so auth/transactional reputation is isolated.
- **Enforce DMARC**. Start at `p=none` to monitor, then move to `p=quarantine` → `p=reject` once SPF
  - DKIM are clean. (Cinder can stay at `p=none` ; BYB should progress to enforcement.)
- **Two separate keys** (5a) — Supabase-Auth vs app-invites — each rotatable without taking the other down. Never commit keys; store only in Railway/Supabase secrets. No shared/demo keys in prod.
- **Rotate on exposure**, and scope keys to **Sending access** only (never full-access).
- Real invites must NOT go out from the console transport — verify the Resend path end-to-end before inviting any real client.

---

## Appendix — using Resend for *product* emails (later)

---

Right now Cinder's only outbound email is **Supabase Auth** (OTP/invite/confirm). The `signup-intake` edge function just writes lead+contact rows — it sends **no** email, and the `automation-runner` logs `in_app` messages, not email.

When you later want Cinder itself to email (e.g. an automation step that emails a lead, or a "your invite was accepted" notice), call the **Resend HTTP API** directly from an edge function — don't route product email through Supabase Auth SMTP:

```
// inside a supabase/functions/* edge function
const r = await fetch("https://api.resend.com/emails", {
  method: "POST",
  headers: {
    Authorization: `Bearer ${Deno.env.get("RESEND_API_KEY")}`,
    "Content-Type": "application/json",
  },
  body: JSON.stringify({
    from: "Cinder <hello@mail.cinder.app>",
    to: [contactEmail],
    subject: "...",
    html: "...",
  }),
});
```

Set the key as a function secret: `supabase secrets set RESEND_API_KEY=re_... --project-ref swxrtifmygripfxdemh`. Use a **separate** API key from the Auth SMTP one so you can rotate/revoke independently.